

✓ Listas

Em programação, é muito comum precisarmos trabalhar com vários dados relacionados ao mesmo tempo. Imagine, por exemplo, que queremos armazenar as notas de um aluno, os nomes de participantes de uma equipe ou ainda uma sequência de medições realizadas por um sensor. Criar uma variável para cada valor rapidamente se torna algo pouco prático e difícil de manter. Para resolver esse tipo de situação, as linguagens de programação oferecem estruturas que permitem armazenar vários valores em uma única variável.

Em Python, uma das estruturas mais utilizadas para esse propósito é a lista. As listas permitem agrupar diversos elementos em uma única estrutura, mantendo uma ordem entre eles. Cada elemento da lista pode ser acessado através de um índice, que indica sua posição dentro da sequência. Dessa forma, conseguimos armazenar, acessar e manipular conjuntos de dados de forma muito mais organizada.

Outra característica importante das listas em Python é a sua flexibilidade. Elas podem armazenar diferentes tipos de dados, como números, textos e até mesmo outras listas. Além disso, listas são estruturas mutáveis, o que significa que podemos modificar seus elementos, adicionar novos valores ou remover itens conforme necessário ao longo da execução do programa.

Nesta aula, iremos aprender o que são listas em Python, como criá-las e como acessar seus elementos. Também veremos como modificar listas, adicionar e remover valores e utilizar algumas operações comuns que facilitam a manipulação de conjuntos de dados. Ao final, você será capaz de utilizar listas para organizar e trabalhar com múltiplas informações dentro dos seus programas.

✓ Declaração de uma Lista

Para criar uma lista, basta colocar a sequência de elementos que deseja adicionar entre `[` e `]` e separados por `,`. Veja abaixo:

```
lista_num = [10, 20, 30, 40, 50]
print("Tipo:", type(lista_num))
print("Lista:", lista_num)

print("\n")

lista_nomes = ["Zezé", "Tim", "Toms"]
print("Tipo:", type(lista_nomes))
print("Lista:", lista_nomes)

print("\n")

lista_floats = [19.4, 15.2, 25.03]
print("Tipo:", type(lista_floats))
print("Lista:", lista_floats)
```

```
Tipo: <class 'list'>
Lista: [10, 20, 30, 40, 50]
```

```
Tipo: <class 'list'>
Lista: ['Zezé', 'Tim', 'Toms']
```

```
Tipo: <class 'list'>
Lista: [19.4, 15.2, 25.03]
```

✓ Acessando elementos e Slicing

Similarmente como vimos na aula de `strings`, as listas podem ser acessadas pelo o index do elemento, bem como podemos usar as estratégias de slicing para obter sub listas.

```
# Acessando Elementos
lista = [10, 20, 30, 40, 50]

# Acessando elemento por index
print(lista[0])

# Acessando o ultimo elemento da lista
print(lista[-1])

# Acessando sub lista dentro da lista
print(lista[2:4])

# Acessando os primeiros 2 elementos da lista
print(lista[:2])
```

```
# Acessando os elementos a partir do 3
print(lista[3:])

# A partir dos 3 ultimos
print(lista[-3:])

# Cortando os 3 ultimos
print(lista[:-2])
```

```
10
50
[30, 40]
[10, 20]
[40, 50]
[30, 40, 50]
[10, 20, 30]
```

▼ Lista de Listas, ou seja, matrizes

Como listas são sequências capazes de armazenar elementos de qualquer tipo, também é possível criar listas que contêm outras listas como elementos. Essa estrutura é conhecida como lista de listas. Cada elemento da lista principal passa a ser, por sua vez, uma nova lista.

Quando organizamos essas listas internas de forma estruturada, com o mesmo número de elementos, podemos representar algo semelhante a uma matriz, muito utilizada em matemática e computação para organizar dados em linhas e colunas.

```
# Matrizes

matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

print("Tipo:", type(matrix))
print("Lista:", matrix)
print("Elemento (1,2):", matrix[1][2])
```

```
Tipo: <class 'list'>
Lista: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
Elemento (1,2): 6
```

```
# Matriz
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

# Acessando uma linha da matriz
linha = matrix[1]

print(type(linha), linha)
```

```
<class 'list'> [4, 5, 6]
```

▼ Operações

Sendo lista uma sequência de dados, podemos realizar algumas operações, idênticas as que fazemos com as strings. Sendo elas:

- Concatenação
- Repetição
- Checagem de elementos

```
lista1 = [1, 2, 3, 4, 5, 6]
lista2 = [7, 8, 9, 10, 11, 12]

# Concatenação
lista_cat = lista1 + lista2
print(lista_cat)

# Repetição da lista
lista_multi = lista1 * 3
print(lista_multi)

# Pertencimento
print(4 in lista1)
print(4 in lista2)
print(4 not in lista2)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
[1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6]
True
False
True
```

✓ Adicionando elementos em uma Lista

Existem varias formas de adicionar elementos em uma lista já existente:

- **Append:** Adicionar um elemento como um novo item a lista
- **Extend:** Concatena elementos na nova lista extendendo ela
- **Insert:** Adiciona um novo elemento na posição desejada

```
# Adicionando elementos
lista = [10, 20, 30, 40, 50]

# Adicionando no final da Lista
lista.append(10)
print(lista)

# Adicionando outra lista concatenando elas
lista.extend([60, 70, 80, 90, 100])
print(lista)

# Adiciona um novo elemento em determinada posição
lista.insert(1, 10)
print(lista)

[10, 20, 30, 40, 50, 10]
[10, 20, 30, 40, 50, 10, 60, 70, 80, 90, 100]
[10, 10, 20, 30, 40, 50, 10, 60, 70, 80, 90, 100]
```

✓ Append vs Extend

Nesta seção vamos entender a diferença entre os métodos `append()` e `extend()`, que são utilizados para adicionar elementos em listas. O método `append()` adiciona um único elemento ao final da lista, mesmo que esse elemento seja outra lista. Já o método `extend()` adiciona cada elemento de uma sequência separadamente à lista original. Compreender essa diferença é importante para evitar erros comuns ao combinar listas e para manipular estruturas de dados de forma correta.

Primeiro vamos ver um caso em que eles fazem a mesma coisa:

```
lista = [10, 20, 30, 40, 50]

lista.append(10)
print(lista)

lista = [10, 20, 30, 40, 50]
lista.extend([10])
print(lista)

[10, 20, 30, 40, 50, 10]
[10, 20, 30, 40, 50, 10]
```

Agora vem a principal diferença, no caso de que o elemento que queremos adicionar se trata de uma lista

```
# Listas
lista1 = [10, 20, 30, 40, 50]
lista2 = [10, 30]

print(lista1)
print(lista2)

# Append
lista1.append(lista2)
print(lista1)

# Extend
lista1 = [10, 20, 30, 40, 50]
lista1.extend(lista2)
print(lista1)

[10, 20, 30, 40, 50]
[10, 30]
```

```
[10, 20, 30, 40, 50, [10, 30]]
[10, 20, 30, 40, 50, 10]
```

Pode se observar que na penúltima linha, o último elemento se trata da `lista2` em si. Já na ultima linha podemos ver que o elementos da `lista2` foram adicionados como elementos da `lista1`.

Podemos observar a importância de saber essa diferença quando trabalhamos com listas de listas. Veja o exemplo abaixo, temos uma lista de coordenadas, e queremos adicionar mais um ponto nessa lista:

```
coords = [
    [1.2, 0.2],
    [0.12, 0.6],
    [4.7, 6.7],
    [11.0, 10.0]
]
```

```
coords1 = coords.copy()
coords2 = coords.copy()
```

```
new_coord = [55.0, 34.4]
```

```
coords1.extend(new_coord)
print("Usando Extend: ", coords1)
```

```
coords2.append(new_coord)
print("Usando Append", coords2)
```

```
Usando Extend: [[1.2, 0.2], [0.12, 0.6], [4.7, 6.7], [11.0, 10.0], 55.0, 34.4]
Usando Append [[1.2, 0.2], [0.12, 0.6], [4.7, 6.7], [11.0, 10.0], [55.0, 34.4]]
```

✓ Removendo Elementos

Nessa seção vamos ver como podemos remover elementos de uma lista. Seja para utiliza-los depois ou para somente remove-los da lista.

- **Pop:** O método `pop` de uma lista retira o elemento presente na posição (index) escolhido e o retorna para que possa ser passado para outra variável.

```
lista = [10, 20, 30, 40, 50, 10]
print("Lista Antes: ", lista)

idx = 1
item = lista.pop(idx)

print("Lista Depois:", lista)
print(f"Item na posição {idx} foi removido, possui valor de {item}")
```

```
Lista Antes: [10, 20, 30, 40, 50, 10]
Lista Depois: [10, 30, 40, 50, 10]
Item na posição 1 foi removido, possui valor de 20
```

- **Remove:** Remove a primeira ocorrência do valor escolhido da esquerda para a direita.

```
lista = [10, 20, 30, 40, 50, 10]
print("Lista Antes: ", lista)

lista.remove(10)
print("Lista Depois:", lista)
```

```
Lista Antes: [10, 20, 30, 40, 50, 10]
Lista Depois: [20, 30, 40, 50, 10]
```

- **Clear:** Esse é o mais direto, ele simplesmente apaga todos elementos da lista.

```
lista = [10, 20, 30, 40, 50, 10]
print("Lista Antes: ", lista)

lista.clear()
print("Lista Depois:", lista)
```

```
Lista Antes: [10, 20, 30, 40, 50, 10]
Lista Depois: []
```

Operações e Funções Úteis

Nessa seção vamos ver algumas funções úteis para utilizar com listas. Veja elas abaixo:

```
lista = [10, 30, 20, 40, 50, 15]
```

```
print("Comprimento:", len(lista)) # Quantos elementos a lista tem
print("Soma:", sum(lista))         # A soma de todos os elementos
print("Max:", max(lista))          # O elemento de maior valor
print("Min:", min(lista))          # O elemento de menor valor
```

```
Comprimento: 6
Soma: 165
Max: 50
Min: 10
```

OBSERVAÇÃO: `sum()`, `max()` e `min()` só funcionam se a lista contém operadores numéricos

```
lista = [1.2, 3, 3 + 2j]
print(sum(lista))
```

```
(7.2+2j)
```

```
lista = ["Zé", 1]
print(sum(lista))
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[65], line 2
      1 lista = ["Zé", 1]
----> 2 print(sum(lista))

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Sorted

Essa função consegue organizar lista de forma crescente ou decrescente (usando o parâmetro `reverse=True`)

```
lista = [10, 30, 20, 40, 50, 15]

print("\nLista anterior:", lista)
print("Lista Sorted:", sorted(lista))

print("\nLista anterior:", lista)
print("Lista Sorted:", sorted(lista, reverse=True))
```

```
Lista anterior: [10, 30, 20, 40, 50, 15]
Lista Sorted: [10, 15, 20, 30, 40, 50]
```

```
Lista anterior: [10, 30, 20, 40, 50, 15]
Lista Sorted: [50, 40, 30, 20, 15, 10]
```

```
lista = ["Zezé", "Tim", "Toms"]

print("\nLista anterior:", lista)
print("Lista Sorted:", sorted(lista))

print("\nLista anterior:", lista)
print("Lista Sorted:", sorted(lista, reverse=True))
```

```
Lista anterior: ['Zezé', 'Tim', 'Toms']
Lista Sorted: ['Tim', 'Toms', 'Zezé']
```

```
Lista anterior: ['Zezé', 'Tim', 'Toms']
Lista Sorted: ['Zezé', 'Toms', 'Tim']
```

Criando Listas com For

No python algumas vezes queremos criar listas que seguem certo padrão. Seja um formato padrão de uma string ou uma conta simples. Poderíamos criar essas listas da seguinte forma:

```
lista = [] # Criando uma lista vazia

# Iterando pelos possiveis nomes de pastas
for i in range(10):
    lista.append(f"pasta_{i}")

# Exibindo lista
print(lista)
```

```
['pasta_0', 'pasta_1', 'pasta_2', 'pasta_3', 'pasta_4', 'pasta_5', 'pasta_6', 'pasta_7', 'pasta_8', 'pasta_9']
```

Mas existe uma forma mais ideal e elegante para se criar esses tipos de listas. Abaixo você pode ver o mesmo exemplo usando *List Comprehension* (Compreensão de lista). Nela conseguimos escrever uma uma linha o mesmo processo de criação da lista.

```
# Criando lista com for
lista = [f"pasta_{i}" for i in range(10)]
print(lista)
```

```
['pasta_0', 'pasta_1', 'pasta_2', 'pasta_3', 'pasta_4', 'pasta_5', 'pasta_6', 'pasta_7', 'pasta_8', 'pasta_9']
```

A estrutura geral para a criação dela é:

```
[expressao for item in iteravel]
```

Também é possível adicionar uma condição:

```
[expressao for item in iteravel if condicao]
```

```
# Criando lista com for
lista = [f"pasta_{i}" for i in range(10) if i % 2 == 0] # Com condição de somente pares
print(lista)
```

```
['pasta_0', 'pasta_2', 'pasta_4', 'pasta_6', 'pasta_8']
```

OBSERVAÇÃO: A expressão `if i % 2 == 0` analisa se o resto da divisão por dois é igual a zero, indicando que o número é par. Esse é o método mais comum de se analisar isso.

Abaixo um exemplo utilizando uma expressão matemática:

```
x_space = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

lista = [x**2 for x in x_space]
print(lista)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

✓ Como usar lista em loops

A Lista é um Data Type iteravel, assim como uma string. Da mesma forma que conseguimos iterar por uma string printando os caracteres que tem nela, podemos iterar por uma lista para analisar ou processar os elementos dela.

- Iterando por elementos

A primeira maneira que vamos ver é o laço sobre um iteravel. Nesse caso, a cada repetição do for o próximo item da lista de `professores` vai passar pra variavel `prof`

```
professores = ["Zezé", "Tim", "Toms"]

for prof in professores:
    print(prof)
```

```
Zezé
Tim
Toms
```

O mesmo ocorre com números

```
numeros = [2, 4, 15, 2, 35, 67, 23]

for num in numeros:
```

```

num += 1
print(f"num + 1 = {num}")

print("Lista final:", numeros)

```

```

num + 1 = 3
num + 1 = 5
num + 1 = 16
num + 1 = 3
num + 1 = 36
num + 1 = 68
num + 1 = 24
Lista final: [2, 4, 15, 2, 35, 67, 23]

```

Mas como podemos ver, não conseguimos alterar os valores da lista dessa forma. Para altera precisamos acessar a lista via index. Sabendo disso, podemos utilizar a função `len()` que retorna o tamanho da lista e usar o `range()`, assim criamos um novo iteravel em que `i` vai ser os index do vetor `numeros`.

```

numeros = [2, 4, 15, 2, 35, 67, 23]

for i in range(len(numeros)):
    numeros[i] += 1

print(numeros)

```

```
[3, 5, 16, 3, 36, 68, 24]
```

Se quisermos acessar tanto o índice quanto o valor de cada elemento da lista durante a iteração, podemos utilizar a função `enumerate()`.

```

notas = [10.0, 9.9, 8.7, 6.8]

for i, nota in enumerate(notas):
    print(f"Index: {i}, Nota: {nota}")

```

```

Index: 0, Nota: 10.0
Index: 1, Nota: 9.9
Index: 2, Nota: 8.7
Index: 3, Nota: 6.8

```

```

lista1 = [0, 1, 3, 4, 5, 6, 7]
lista2 = [x**2 for x in lista1]

print("Lista1:", lista1)
print("Lista2:", lista2)

for lado, area in zip(lista1, lista2):
    print(f"Lado: {lado}, Area: {area}")

```

```

Lista1: [0, 1, 3, 4, 5, 6, 7]
Lista2: [0, 1, 9, 16, 25, 36, 49]
Lado: 0, Area: 0
Lado: 1, Area: 1
Lado: 3, Area: 9
Lado: 4, Area: 16
Lado: 5, Area: 25
Lado: 6, Area: 36
Lado: 7, Area: 49

```

▼ Exemplo

Agora vamos ver um exemplo de aplicação. Na ultima aula vimos uma simulação que não utiliza um vetor de tempo. Podemos ver uma outra abordagem usando uma lista que representa cada instante de tempo.

```

dt = 0.005
duration = 5.0

# Lista temporal
time = [dt * i for i in range(int(duration/dt))]

# Condições iniciais
x = 0
dx = 0
ddx = 0.5

# Simulação

```

```
for i, t in enumerate(time):  
  
    # calculando novo estado  
    dx_new = dx + ddx * dt  
    x_new = x + dx_new * dt  
  
    # printar apenas em algumas iterações  
    if i % 100 == 0:  
        print(f"t: {t:.3f} x: {x_new:.2f}, dx: {dx_new:.2f}")  
  
    # atualizando estado  
    x = x_new  
    dx = dx_new
```

```
t: 0.000 x: 0.00, dx: 0.00  
t: 0.500 x: 0.06, dx: 0.25  
t: 1.000 x: 0.25, dx: 0.50  
t: 1.500 x: 0.57, dx: 0.75  
t: 2.000 x: 1.01, dx: 1.00  
t: 2.500 x: 1.57, dx: 1.25  
t: 3.000 x: 2.26, dx: 1.50  
t: 3.500 x: 3.08, dx: 1.75  
t: 4.000 x: 4.02, dx: 2.00  
t: 4.500 x: 5.08, dx: 2.25
```